

DOCUMENTATION DU PROJET

1. Introduction

Le projet PasswordGeneratorCLI est une application en ligne de commande développée en Java 21. Elle permet de générer des mots de passe sécurisés selon plusieurs critères définis par l'utilisateur et d'évaluer leur robustesse à l'aide d'un outil externe exécuté dans un conteneur Docker.

Ce projet s'inscrit dans une démarche DevOps, combinant une application Java avec une analyse de sécurité externalisée. L'objectif principal est de simuler un environnement professionnel où une application communique avec un service isolé pour effectuer des traitements spécialisés.

2. Analyse fonctionnelle

2.1 Objectif de l'application

L'application a pour objectif de générer des mots de passe sécurisés et personnalisables, tout en fournissant une évaluation de leur niveau de sécurité.

2.2 Fonctionnalités principales

Génération de mots de passe

L'utilisateur peut générer un mot de passe en définissant :

- la longueur du mot de passe
- les types de caractères utilisés :
 - lettres minuscules
 - lettres majuscules
 - chiffres
 - symboles

Mode rafale

Le programme permet également de générer plusieurs mots de passe en une seule exécution, en fonction d'un paramètre de répétition.

Analyse de robustesse

Chaque mot de passe généré est évalué selon un score de sécurité retourné sous forme de niveau :

- Très faible
- Faible
- Moyen
- Fort
- Très fort

Cette analyse est réalisée par un outil externe exécuté dans un conteneur Docker.

2.3 Interface utilisateur

L'application fonctionne exclusivement en ligne de commande (CLI). Les paramètres sont fournis lors de l'exécution du programme sous forme d'arguments.

3. Analyse technique

3.1 Architecture générale

Le système est composé de deux parties principales :

- une application Java responsable de la génération des mots de passe
- un conteneur Docker chargé de l'analyse de la robustesse

Le flux de fonctionnement est le suivant :

Java Application → ProcessBuilder → Docker Container → Résultat retourné → Java

3.2 Technologie utilisée

Le projet repose sur les technologies suivantes :

- Java 21 pour la logique principale
- Maven pour la gestion du projet
- Docker pour l'exécution de l'analyse externe
- Node.js pour le script d'analyse dans le conteneur

3.3 Communication Java et Docker

La communication entre Java et Docker est réalisée à l'aide de la classe `ProcessBuilder`.

Le programme Java lance une commande Docker depuis le système et récupère la sortie standard du conteneur.

Exemple de fonctionnement :

- Java exécute une commande Docker
- Docker reçoit le mot de passe en entrée
- Docker retourne un niveau de robustesse
- Java lit le résultat via un flux de sortie

Cette approche permet de découpler l'analyse de sécurité de la logique principale.

3.4 Génération des mots de passe

La génération repose sur un ensemble de caractères définis dans le programme :

- lettres minuscules
- lettres majuscules
- chiffres
- symboles

Un générateur aléatoire basé sur `SecureRandom` est utilisé pour garantir l'unicité et la sécurité des mots de passe.

4. Architecture du projet

Le projet est organisé selon la structure suivante : `PasswordGeneratorCLI/`

- `docker/`
 - `Dockerfile`
 - `app.js`
- `src/`
 - `main/java/ci/univmetiers/passwordgenerator/`
 - `Main.java`
 - `PasswordGenerator.java`
- `pom.xml`
- `README.md`

Cette organisation respecte une architecture standard Maven.

5. Guide d'installation et d'exécution

5.1 Prérequis

Avant d'exécuter le projet, il est nécessaire d'installer :

- Java 21
- Docker Desktop
- Maven (optionnel)

5.2 Construction de l'image Docker

Pour construire l'image Docker, exécuter la commande suivante :

```
cd docker  
docker build -t password-analyzer .
```

5.3 Exécution de l'image Docker

```
docker run password-analyzer "Abc12345!"
```

5.4 Compilation et exécution du projet Java

```
java -cp target/classes ci.univmetiers.passwordgenerator.Main 12 true true true true 3
```

5.5 Description des paramètres

Les paramètres sont les suivants :

- longueur du mot de passe
- inclusion des minuscules (true/false)
- inclusion des majuscules (true/false)
- inclusion des chiffres (true/false)
- inclusion des symboles (true/false)
- nombre de mots de passe à générer

6. Exemples d'exécution

Exemple de sortie possible :

1. A9k#Lm2P → Fort
2. Zx8@LpQ7 → Moyen
3. Wp2!KsT9 → Très fort

Chaque mot de passe est généré de manière aléatoire et évalué par le module Docker.

7. Limites et améliorations

Limites

- dépendance à Docker pour l'analyse
- exécution en ligne de commande uniquement
- absence d'interface graphique

Améliorations possibles

- ajout d'une interface graphique
- amélioration de l'analyse de sécurité
- ajout d'un export de mots de passe
- intégration d'une API externe de sécurité

8. Conclusion

Le projet PasswordGeneratorCLI permet de comprendre la mise en place d'une architecture simple DevOps combinant une application Java et un service Docker.